



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Arief Gunawan
NPM	2428240087
Kelas	SI5B
Topik Tugas 1	Peternakan
Entitas utama	Hewan Ternak
Tautan repositori	https://github.com/Ayep67/si5b_kuis1_ariefgunawan
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	08-10-2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Peternakan
Entitas utama (tunggal)	Livestock
Alamat dasar	/livestock
Field data	id, nama, jenis, umur, berat, sehat
Field wajib saat POST	nama, jenis, umur, berat, sehat
Query string untuk filter	jenis
Data awal	Sapi Limousin, Kambing Etawa, Ayam Broiler
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-daya]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-daya]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-daya]	Ya	201	Tambah data
4	PUT	/[sumber-daya]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-daya]/:id	Ya	204	Hapus data

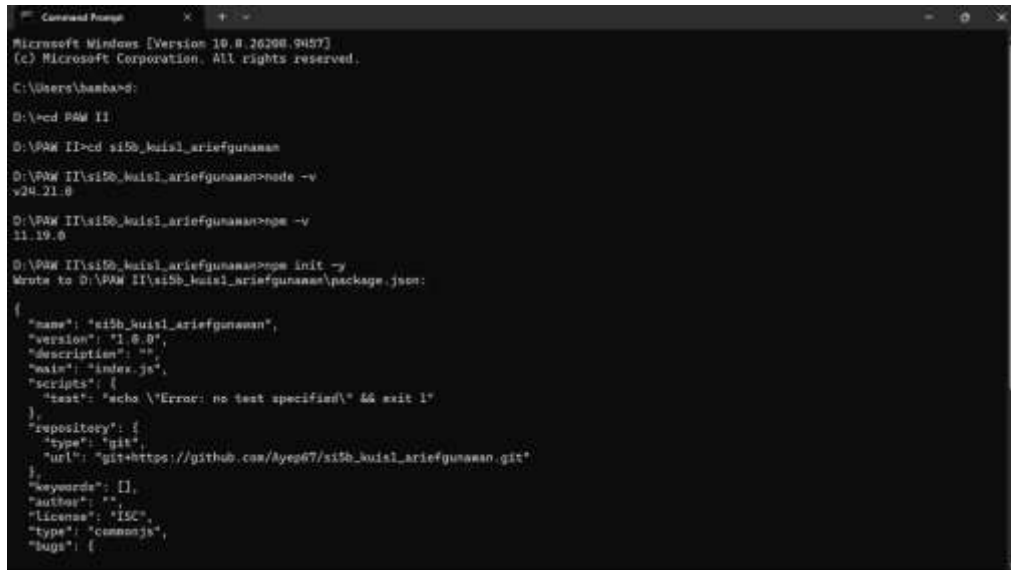
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



```

Microsoft Windows [Version 10.0.26290.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hamba>

D:\>cd PAW II

D:\PAW II>cd si5b_huisl_ariefgunawan

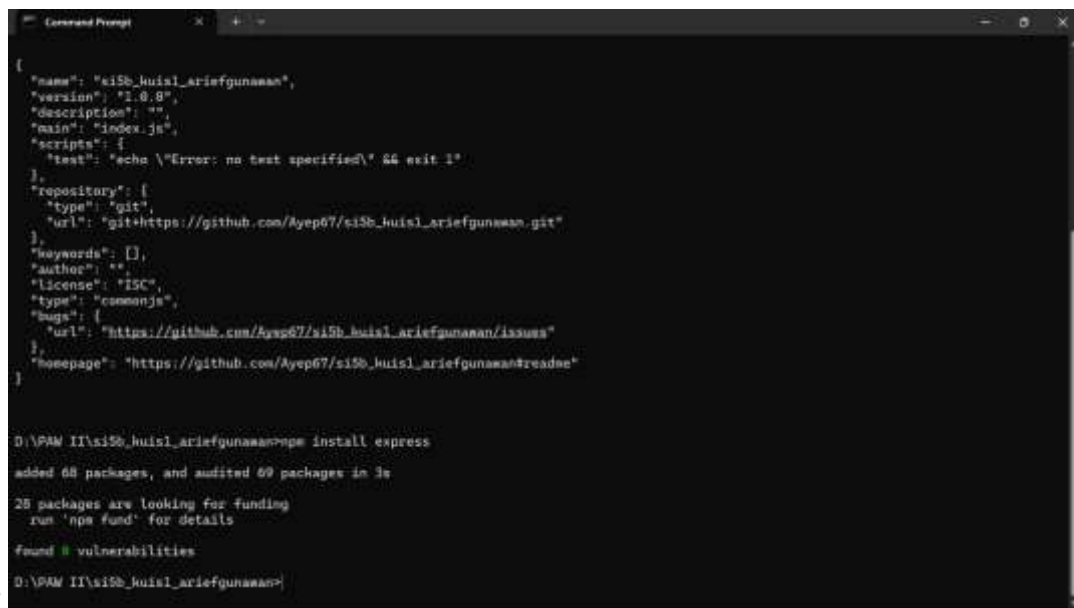
D:\PAW II\si5b_huisl_ariefgunawan>node -v
v24.21.0

D:\PAW II\si5b_huisl_ariefgunawan>npm -v
11.19.0

D:\PAW II\si5b_huisl_ariefgunawan>npm init -y
Wrote to D:\PAW II\si5b_huisl_ariefgunawan\package.json:

{
  "name": "si5b_huisl_ariefgunawan",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/Ayep67/si5b_huisl_ariefgunawan.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {}
}

```



```

{
  "name": "si5b_huisl_ariefgunawan",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/Ayep67/si5b_huisl_ariefgunawan.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/Ayep67/si5b_huisl_ariefgunawan/issues"
  },
  "homepage": "https://github.com/Ayep67/si5b_huisl_ariefgunawan#readme"
}

D:\PAW II\si5b_huisl_ariefgunawan>npm install express
added 68 packages, and audited 69 packages in 3s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\PAW II\si5b_huisl_ariefgunawan>

```

Gambar 1. Versi Node.js dan instalasi paket

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

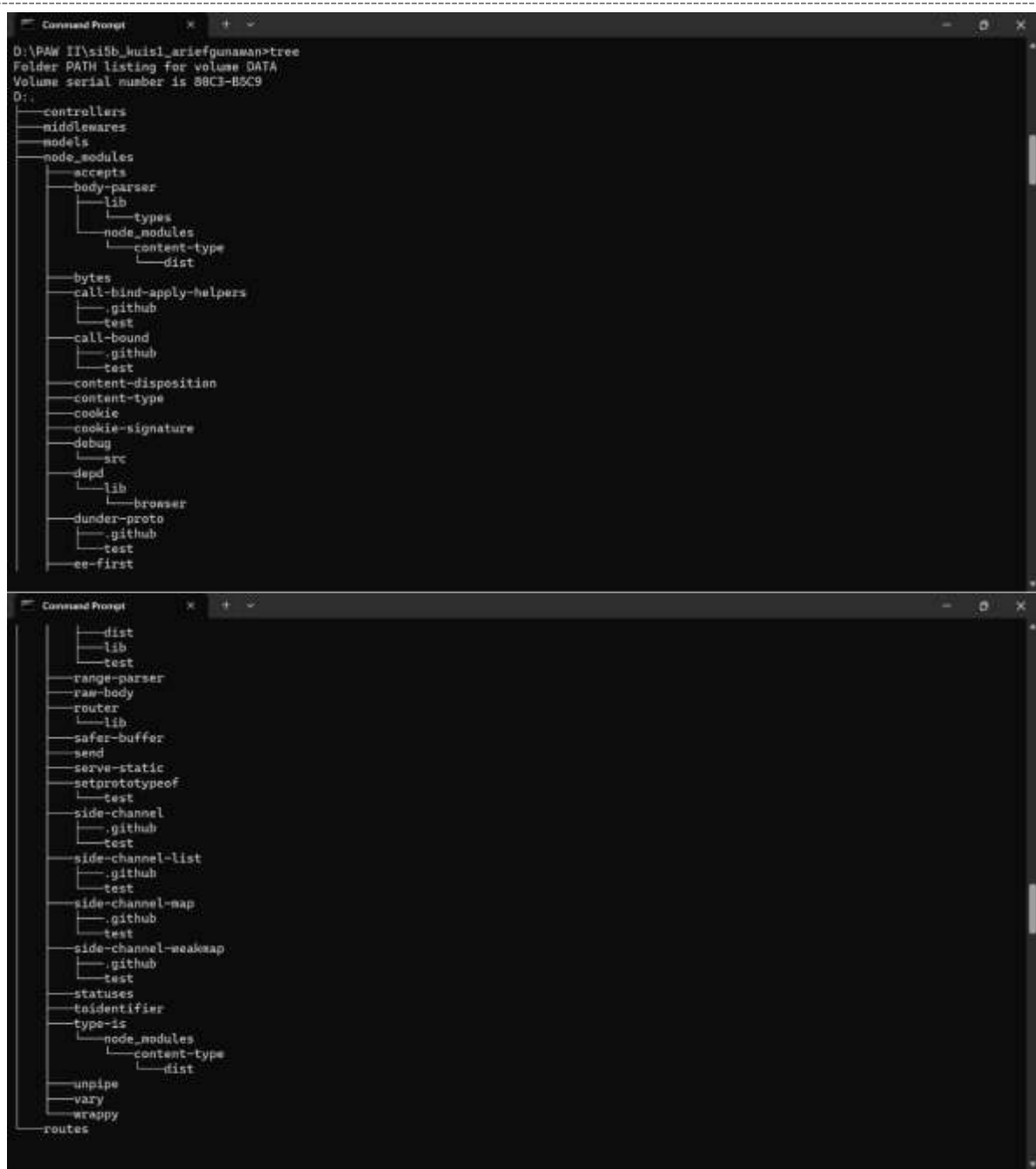
Express.js merupakan framework Node.js yang digunakan untuk membangun aplikasi web dan RESTful API, seperti membuat endpoint dengan metode GET, POST, PUT, dan DELETE serta mengatur request dan response dari server. CORS (Cross-Origin Resource Sharing) berfungsi mengatur izin akses API dari aplikasi atau domain yang berbeda sehingga frontend dapat berkomunikasi dengan backend tanpa terkena pembatasan keamanan browser. Sementara itu, dotenv digunakan untuk membaca konfigurasi dari file .env dan menyimpannya ke process.env, misalnya port server, URL database, atau konfigurasi lainnya, sehingga data konfigurasi tidak perlu ditulis langsung di dalam kode. Versi Node.js perlu memenuhi persyaratan Express 5 karena Express 5 memiliki kebutuhan runtime dan dependency tertentu yang memerlukan versi Node.js yang kompatibel. Penggunaan Node.js yang sesuai, terutama versi LTS yang masih didukung, membantu memastikan Express 5 dapat diinstal, dijalankan, dan menggunakan seluruh fiturnya dengan baik tanpa masalah kompatibilitas..

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Gambar 2. Struktur folder project

Penjelasan:

Routes berfungsi menentukan alamat endpoint dan metode HTTP yang digunakan, seperti GET /livestock atau POST /livestock. **Controllers** berfungsi menangani logika proses ketika endpoint dipanggil, seperti mengambil, menambah, mengubah, atau menghapus data, kemudian mengirimkan response kepada client. **Models** berfungsi merepresentasikan dan mengatur struktur data yang digunakan aplikasi, misalnya data livestock yang memiliki atribut. **Middlewares** menjalankan pemeriksaan atau pemrosesan tambahan sebelum request diteruskan.

2.2 Ringkasan tugas setiap berkas

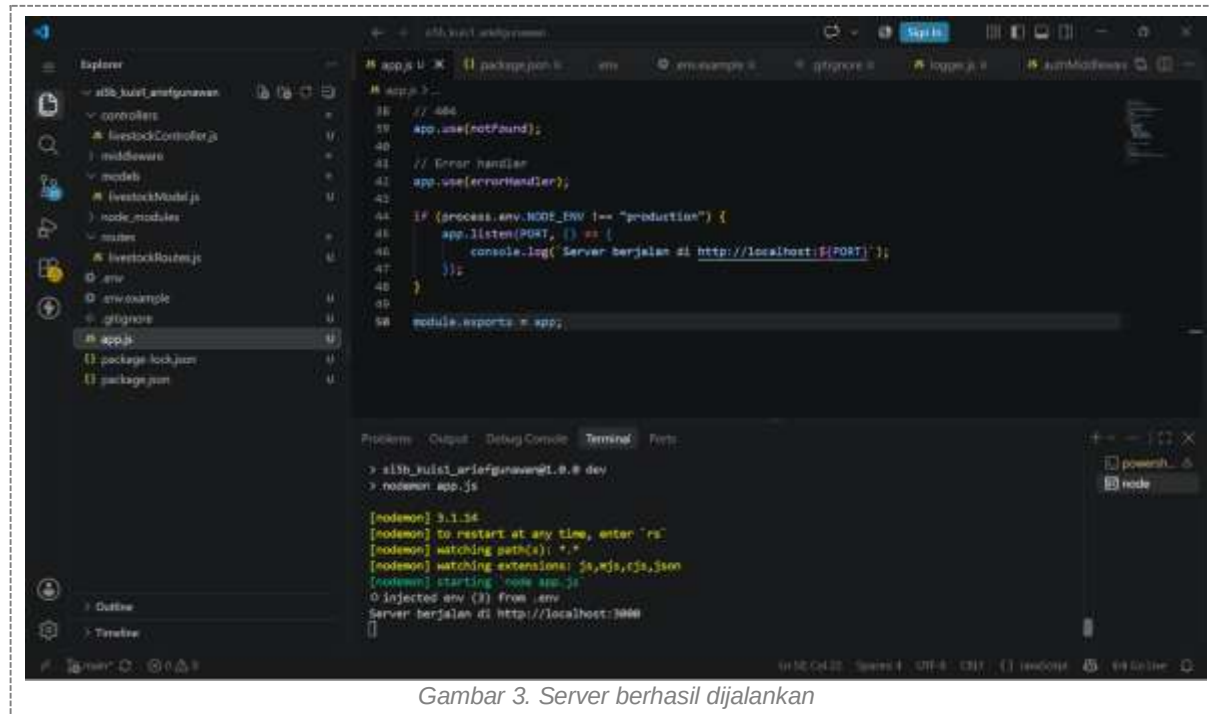
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
Models/livestockModel.js	Menyimpan data awal livestock dalam array serta menyediakan fungsi untuk mengambil
controllers/livestockController.js	Menangani request dari pengguna
Routes/livestockRoutes.js	Mendefinisikan endpoint API menggunakan <code>express.Router()</code> dan menghubungkan setiap alamat endpoint dengan fungsi Controller
middlewares/logger.js	Mencatat setiap request yang masuk ke server
middlewares/cekApiKey.js	Memeriksa API Key pada request melalui header <code>x-api-key</code> .
middlewares/errorHandler.js	Menangani error secara terpusat
app.js	Menjadi file utama aplikasi yang menginisialisasi Express

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

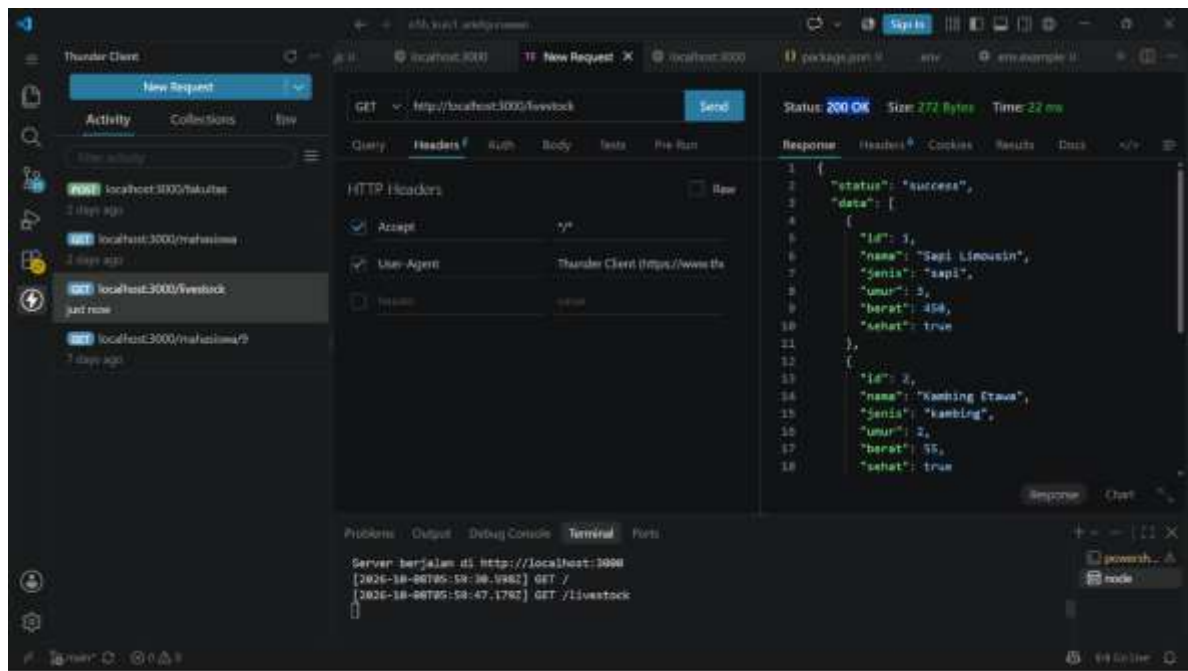
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-hay])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /livestock
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK
Lulus?	[1] Ya [] Tidak



Gambar 4. Hasil GET semua data

Gambar 4. Hasil GET semua data

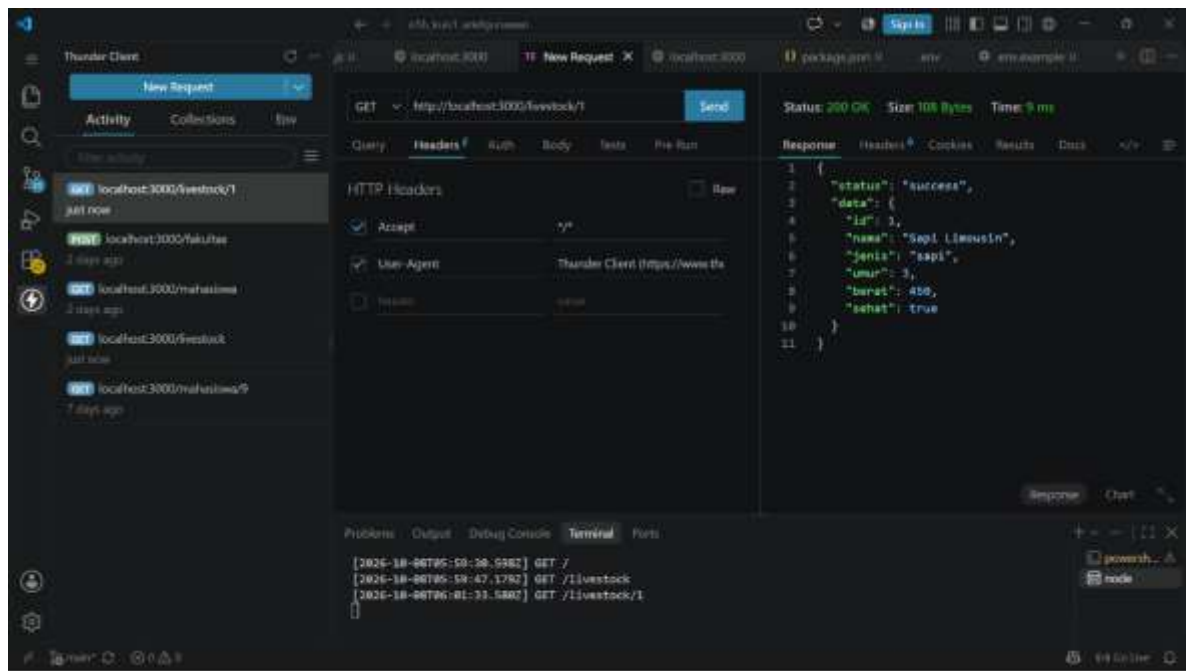
Penjelasan:

Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /livestock/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK
Lulus?	[1] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

Gambar 5. Hasil GET berdasarkan ID

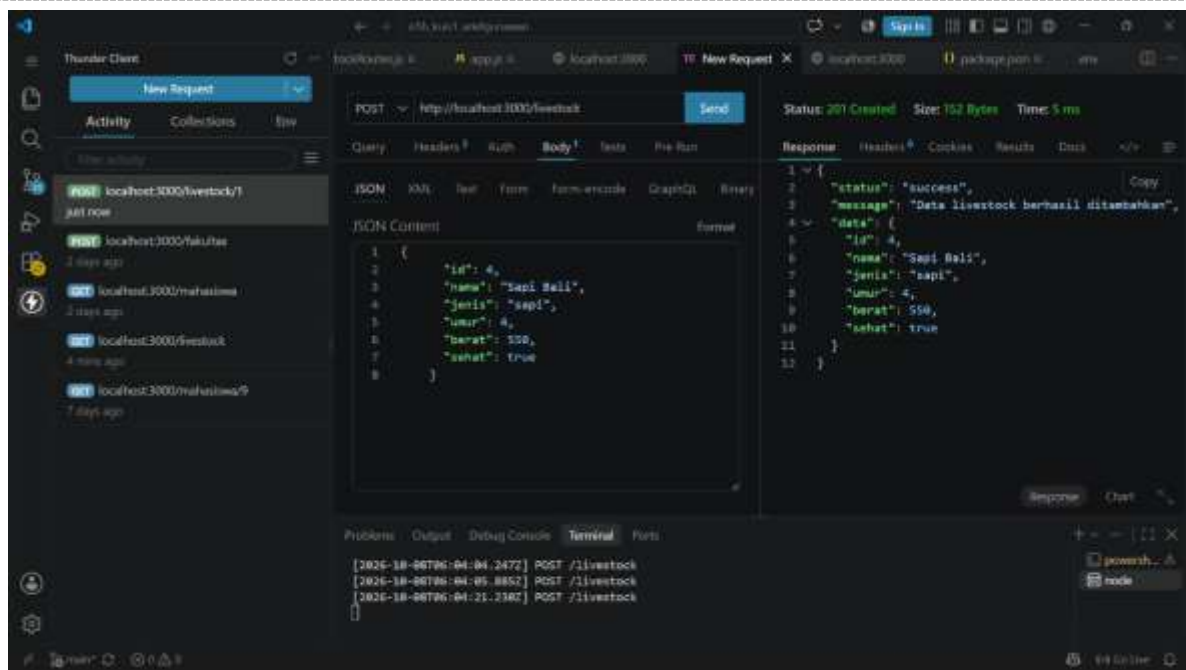
Penjelasan:

Jelaskan bagaimana ID dari alamat diambil (req.params) dan mengapa perlu diubah menjadi angka.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (<i>request</i>)	POST /livestock
Header	x-api-key: (arief-api-2026) Content-Type: application/json
Body	{ "id": 4, "nama": "Sapi Bali", "jenis": "sapi", "umur": 4, "berat": 550, "sehat": true }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created
Lulus?	[1] Ya [] Tidak



Gambar 6. Hasil POST data baru

Gambar 6. Hasil POST data baru

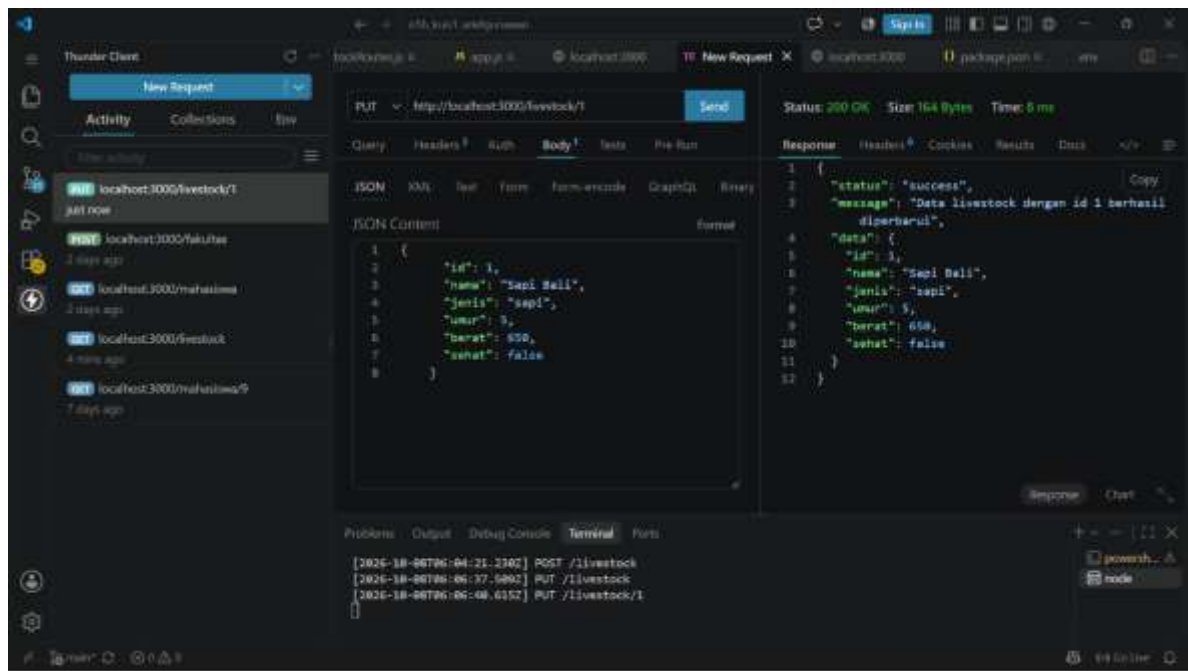
Penjelasan:

Jelaskan peran header `x-api-key` (middleware apa yang memeriksanya), peran `express.json()`, dan alasan status yang dipakai adalah 201, bukan 200.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (<i>request</i>)	PUT /livestock/1]
Header	x-api-key: (arief-api-2026) Content-Type: application/json
Body	{ "id": 1, "nama": "Sapi Bali", "jenis": "sapi", "umur": 5, "berat": 650, "sehat": false }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK
Lulus?	[1] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

Gambar 7. Hasil PUT mengubah data

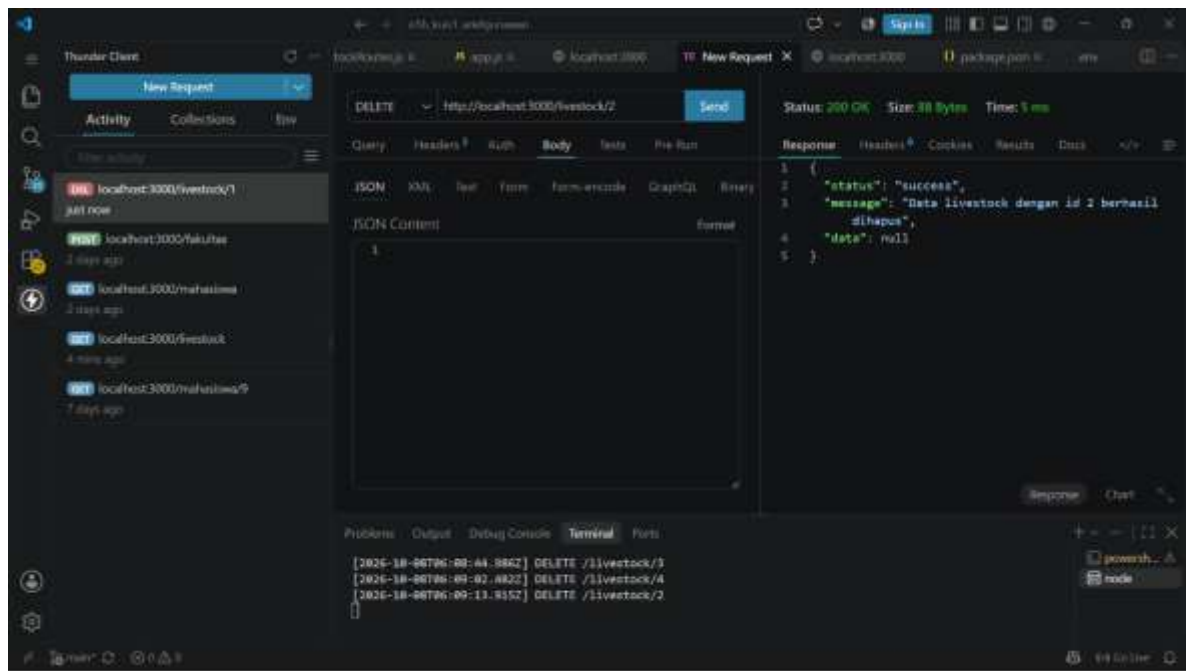
Penjelasan:

Jelaskan field apa yang berubah dan field apa yang tetap, serta layer mana yang melakukan perubahan.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /livestock/2
Header	x-api-key: (arief-api-2026)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	200 OK
Lulus?	[] Ya [1] Tidak



Gambar 8. Hasil DELETE data

Gambar 8. Hasil DELETE data

Penjelasan:

Jelaskan arti status 204 dan mengapa respons tidak berisi apa pun.

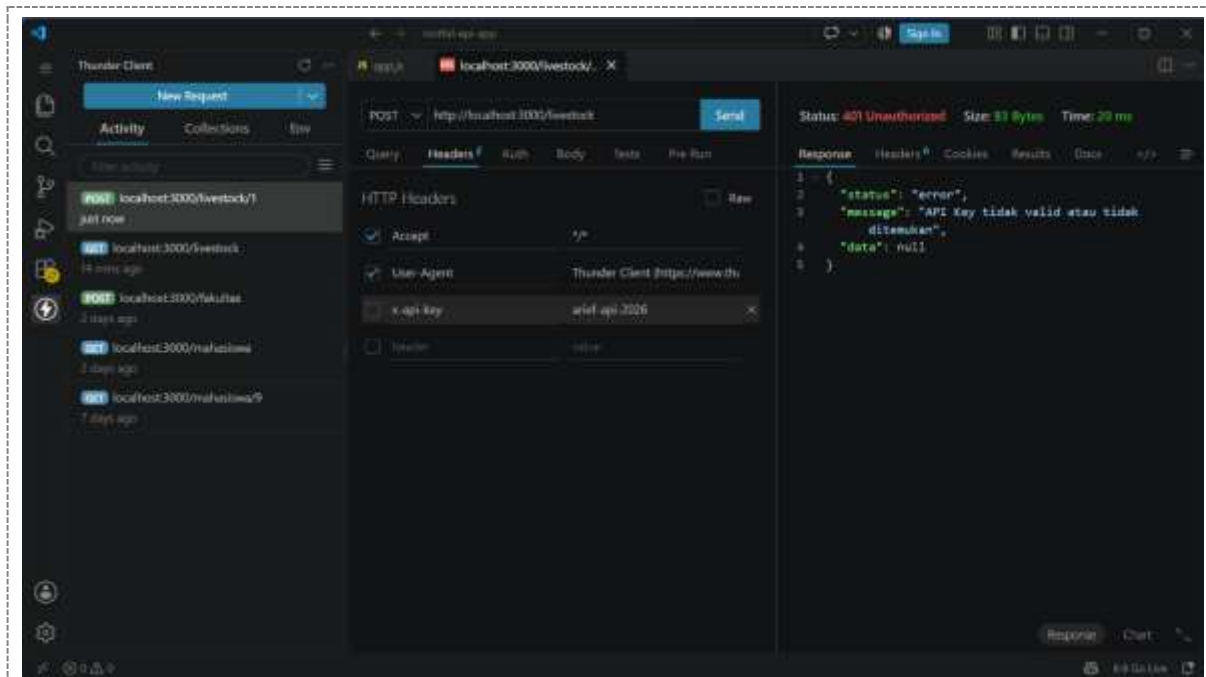
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /livestock (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized
Lulus?	[1] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

Gambar 9. POST tanpa API key ditolak

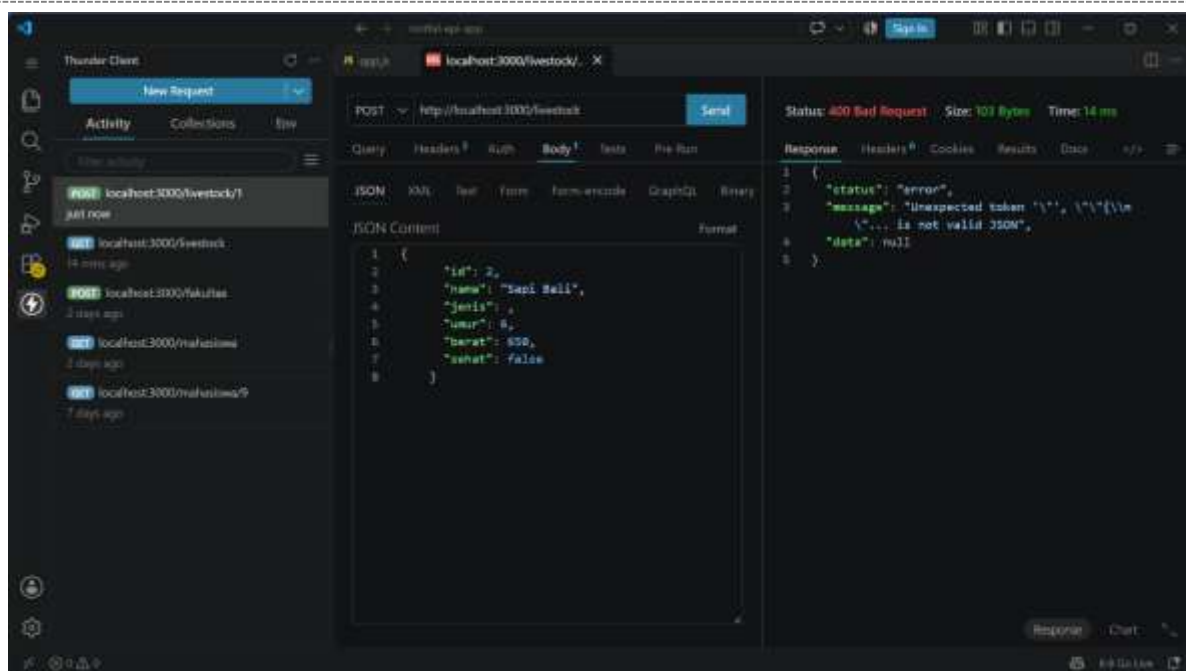
Penjelasan:

Jelaskan middleware mana yang menolak permintaan dan mengapa Controller tidak sempat dijalankan.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /livestock
Header	x-api-key: (arief-api-2026) Content-Type: application/json
Body	{ hanya sebagian field wajib }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request
Lulus?	[1] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

Gambar 10. POST dengan body tidak lengkap

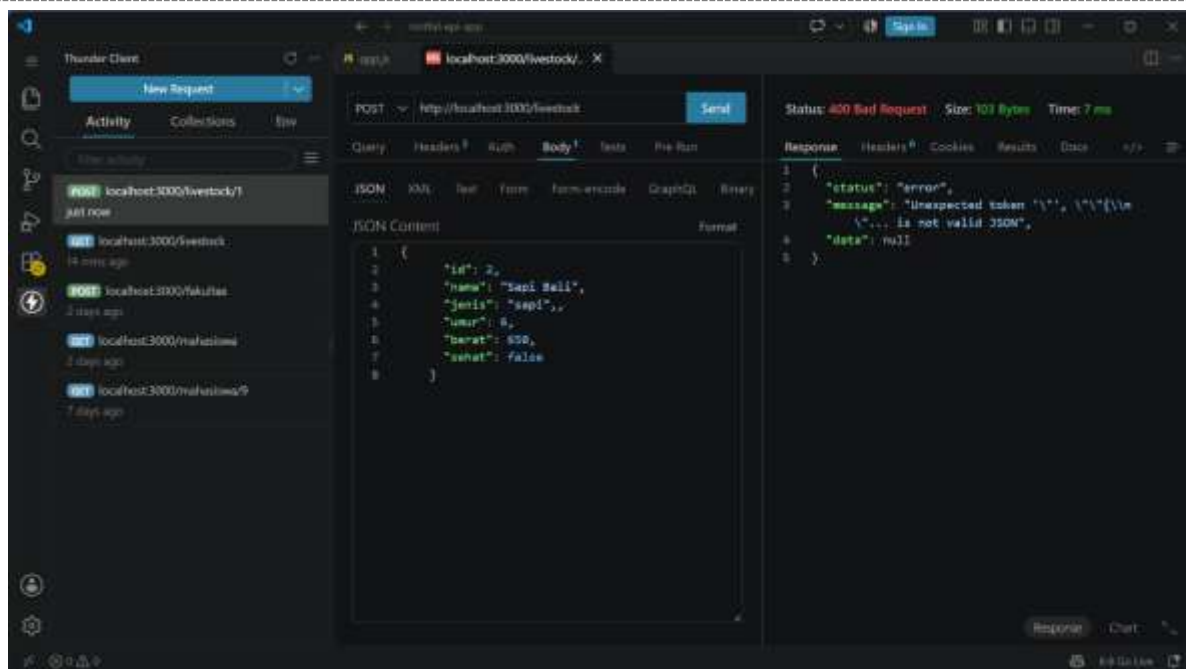
Penjelasan:

Jelaskan di layer mana validasi ini ditulis dan mengapa kode statusnya 400.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /livestock
Header	x-api-key: (arief-api-2026) Content-Type: application/json
Body	{ JSON dengan koma berlebih }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request
Lulus?	[1] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

Gambar 11. POST dengan JSON rusak

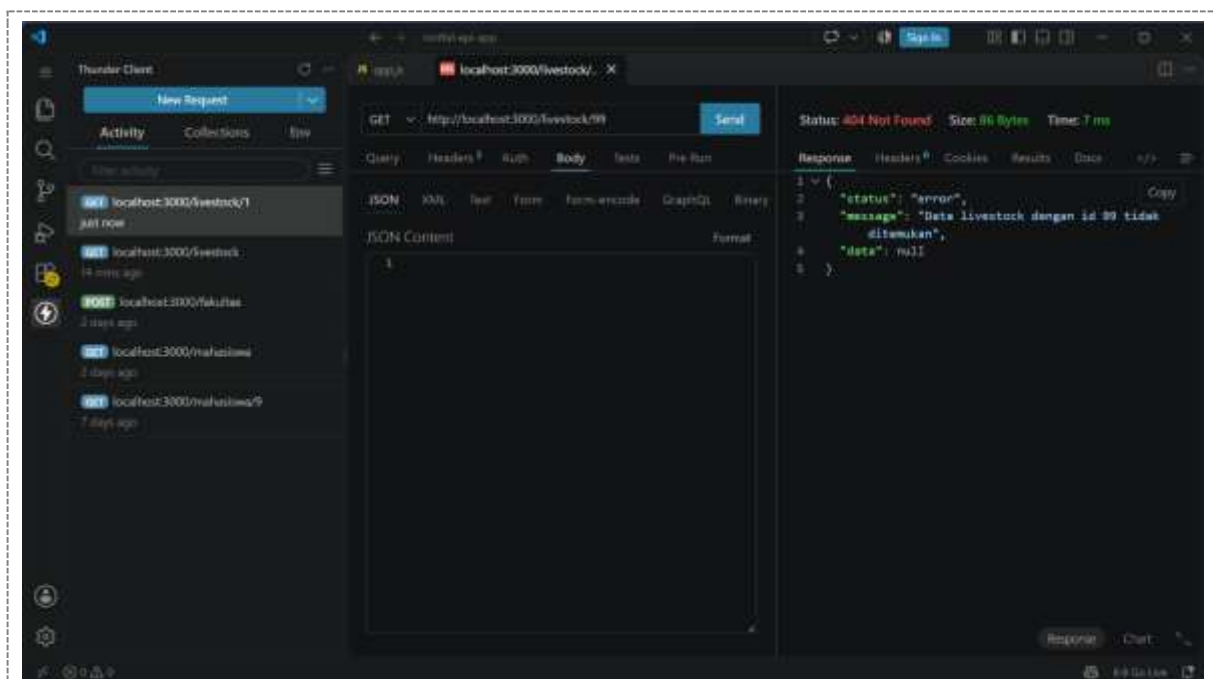
Penjelasan:

Jelaskan siapa yang mendeteksi JSON rusak ini dan bagaimana error handler mengubahnya menjadi respons yang rapi.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /livestock/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found
Lulus?	[1] Ya [] Tidak



Gambar 12. GET ID tidak ada

Gambar 12. GET ID tidak ada

Penjelasan:

Jelaskan alur kode sampai pesan 404 dibuat dan siapa yang mengirimkannya.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200		
T02	Menampilkan satu data berdasarkan ID	200		
T03	Menambah data baru	201		
T04	Mengubah data	200		
T05	Menghapus data	204		
N01	Menambah data tanpa API key	401		
N02	Menambah data dengan body tidak lengkap	400		
N03	Mengirim JSON yang rusak	400		
N04	Mengambil ID yang tidak ada	404		

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	
Pesan commit terakhir	
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Pada kode Livestock, Route bertugas menentukan alamat endpoint dan mengarahkannya ke Controller, contohnya `router.get("/", livestockController.getAll)`, sedangkan Controller menangani request, validasi, dan response, contohnya `const data = livestockModel.getAll(jenis)`. Model bertugas mengelola data dan operasi CRUD tanpa menggunakan req atau res, contohnya `function getAll(jenis) { return livestock.filter(item => item.jenis.toLowerCase() === jenis.toLowerCase()); }`

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Keuntungan error handler terpusat adalah kode menjadi lebih ringkas, konsisten, dan mudah dipelihara karena kesalahan seperti 404 cukup ditangani di satu middleware tanpa menulis `res.status(404)` pada setiap route.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	1
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	1
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	1
4	Identitas saya (nama atau NIM) muncul pada data uji POST	1
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	1
6	Tabel rekapitulasi sesuai dengan tangkapan layar	1
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	1
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	1

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100